WEBSITE
VULNERABILITY ASSESSMENT REPORT

Prepared By

SAMSON BUDIGILA.

A vulnerability assessment performed on google gruyere

Samsonbudigila6@gmail.com

Future interns

0757502743

TABLE OF CONTENTS

# Section	Sub-section
1 Non-Disclosure Statement	
2 Legal Notice	
3 Executive Summary	
4 Introduction	
5 Scope	
6 Risk Rating Classification	
7 Vulnerabilities Overview	
8 Vulnerabilities	
	8.1 Information Disclosure
	8.2 Stored XSS (Cross-Site Scripting)
	8.3 File Upload XSS
	8.4 Absence of Security Headers
	8.5 Cross-Site Request Forgery (CSRF)
	8.6 AJAX-Based Client-Side Manipulation
9 Appendix – Tools Used	

1. Non-Disclosure Statement

This security assessment was conducted solely on **https://google-gruyere.appspot.com/**, a deliberately vulnerable web application developed and maintained by Google for educational and cybersecurity training purposes. All findings, vulnerabilities, and technical details identified during this assessment are strictly confidential and intended exclusively for academic study and personal skill development. The information gathered through this assessment shall not be shared, published, distributed, or disclosed to any unauthorized third party without prior written consent from the relevant stakeholders. No data collected during this assessment will be used for any malicious, unauthorized, or commercial purpose beyond the scope of this cybersecurity study. The assessor acknowledges full responsibility for conducting this evaluation in a legal, ethical, and professional manner, in accordance with the terms of use of the target application and applicable cybersecurity laws and regulations.

2. LEGAL NOTICE

This vulnerability assessment was conducted educational security testing purposes. All assessment activities were performed using non-intrusive, passive, industry-standard tools including OWASP ZAP, Nmap, and Nikto, strictly within the authorized scope of the target domain and in full compliance with ethical penetration testing principles and applicable cybersecurity laws. All findings and technical details identified here are strictly confidential, intended solely for academic evaluation, and shall not be disclosed or distributed to any unauthorized party without prior written consent. No malicious, destructive, or unauthorized techniques were employed at any point during this assessment, and the assessor accepts full personal responsibility for the ethical and legal conduct of all activities performed. This report does not constitute a comprehensive security audit and the assessor shall not be held liable for any damages or consequences arising from the misuse of information contained within this document.

3. EXECUTIVE SUMMARY

This security assessment was conducted by **Samson Budigila** against the deliberately vulnerable web application **Google Gruyere** (https://google-gruyere.appspot.com/), a platform designed and maintained by Google for cybersecurity training and educational purposes. The objective of the assessment was to identify and document exploitable security weaknesses present within the application, assess their severity, and recommend practical remediation measures.

During the assessment, a total of **6 vulnerabilities** were identified and classified using the **DREAD risk rating framework**:

The four **High-risk** vulnerabilities Information Disclosure via an exposed debug file, Stored Cross-Site Scripting (XSS), File Upload XSS, and Cross-Site Request Forgery (CSRF) represent critical threats that could allow an attacker to steal credentials, hijack user sessions, execute arbitrary code in victims' browsers, and perform unauthorized destructive actions without user consent.

The two **Medium-risk** vulnerabilities the complete absence of HTTP security headers and insecure AJAX-based client-side data handling significantly reduce the application's resilience against browser-based attacks such as clickjacking, MIME-sniffing, and request manipulation.

All testing activities were performed using industry-standard tools including **OWASP ZAP**, **Burp Suite**, **Nmap**, and **SecurityHeaders.com**, strictly within the authorized scope of the target domain and in full compliance with ethical penetration testing principles.

4. INTRODUCTION

This report presents the findings of a **Grey-box** vulnerability assessment conducted against Google Gruyere, a purposely insecure web application built by Google as a hands-on cybersecurity learning environment. The assessment was carried out by **Samson Budigila** to identify security-related weaknesses, evaluate their potential impact, and provide actionable countermeasures.

The findings documented in this report cover vulnerabilities identified across the application's authentication mechanism, input handling, file upload functionality, HTTP response headers, session management, and client-side scripting behaviour. Each vulnerability is accompanied by a risk rating, a description of the attack technique used, proof-of-concept evidence, and recommended countermeasures.

5. SCOPE:

Assessment	Target
Vulnerability assessment	https://google-gruyere.appspot.com/

6. RISK RATING CLASSIFICATION.

Vulnerabilities severity are supplied using DREAD framework giving an indication of their severity and are rated on a scale of five (5) to fifteen (15) using the DREAD model below

	VULNERABILITY.	D	R	E	A	D	TOTAL	RATING
1	Information Disclosure	3	3	3	3	2	14	High
2	Stored XSS	3	3	3	3	3	15	High
3	File Upload XSS	3	3	3	3	2	14	High
4	Absence of Security Headers	2	3	2	3	3	13	Medium
5	Cross-Site Request Forgery (CSRF)	3	3	2	3	2	13	High
6	AJAX Client-Side Manipulation	2	2	2	2	2	10	Medium

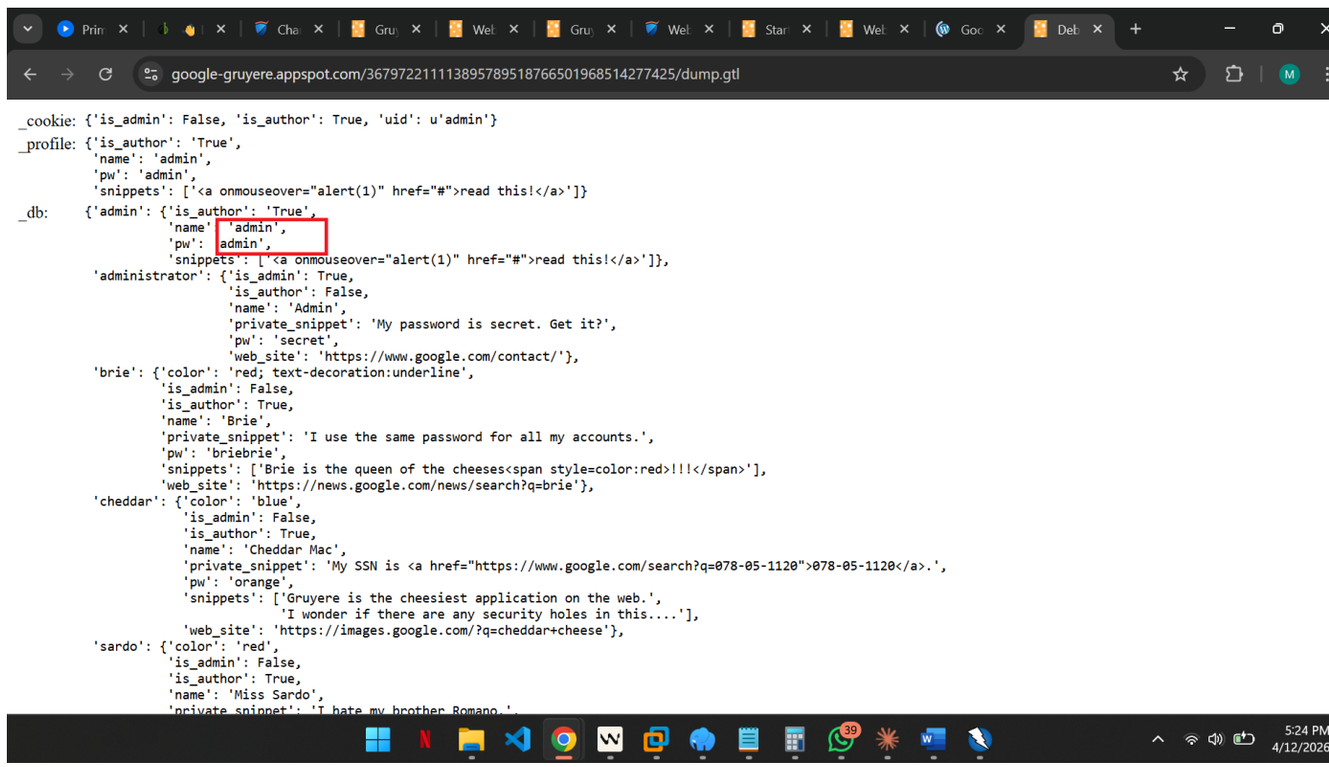
Score range: 5 (lowest) → 15 (critical)

7. VULNERABILITIES OVERVIEW

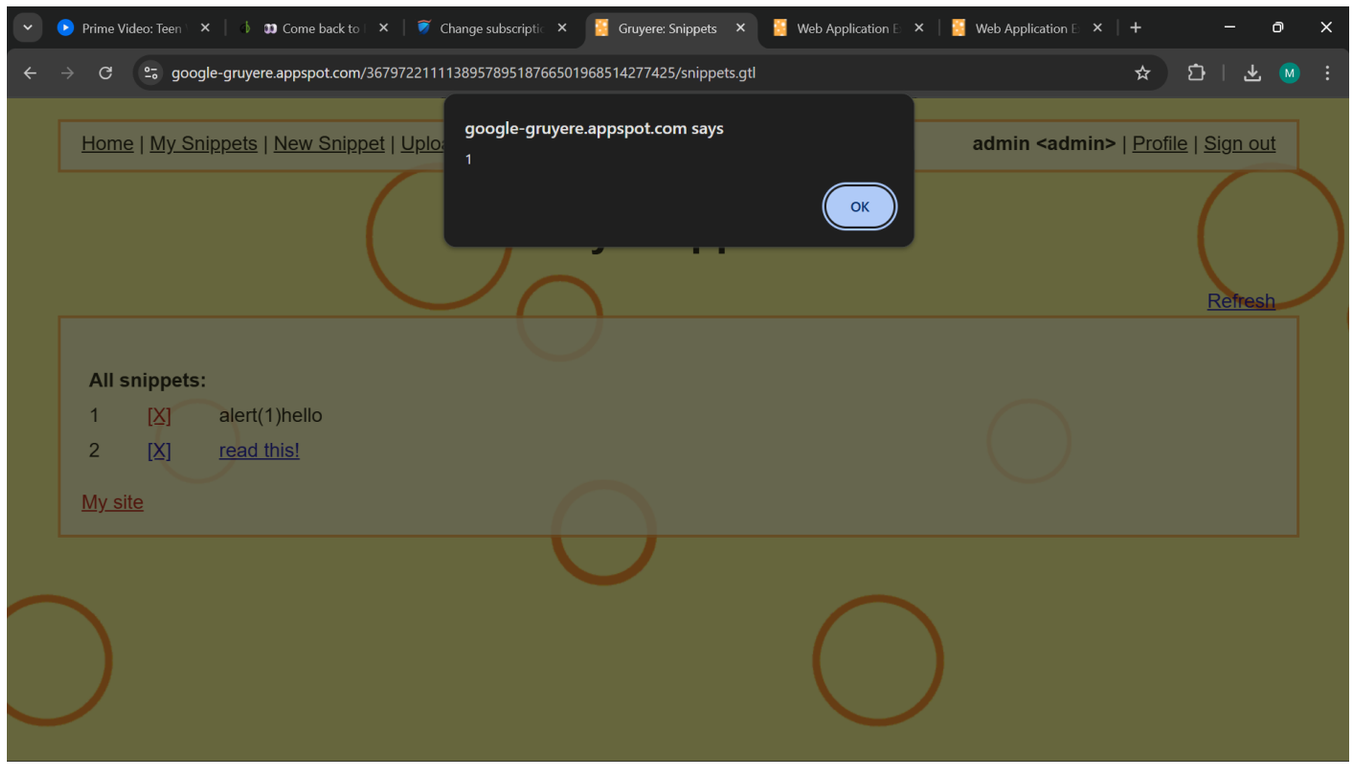
Vulnerabilities grouped by risk severity	
High (3)	HIGH
Medium (2)	MEDIUM
Low (1)	LOW

8. VULNERABILITIES.

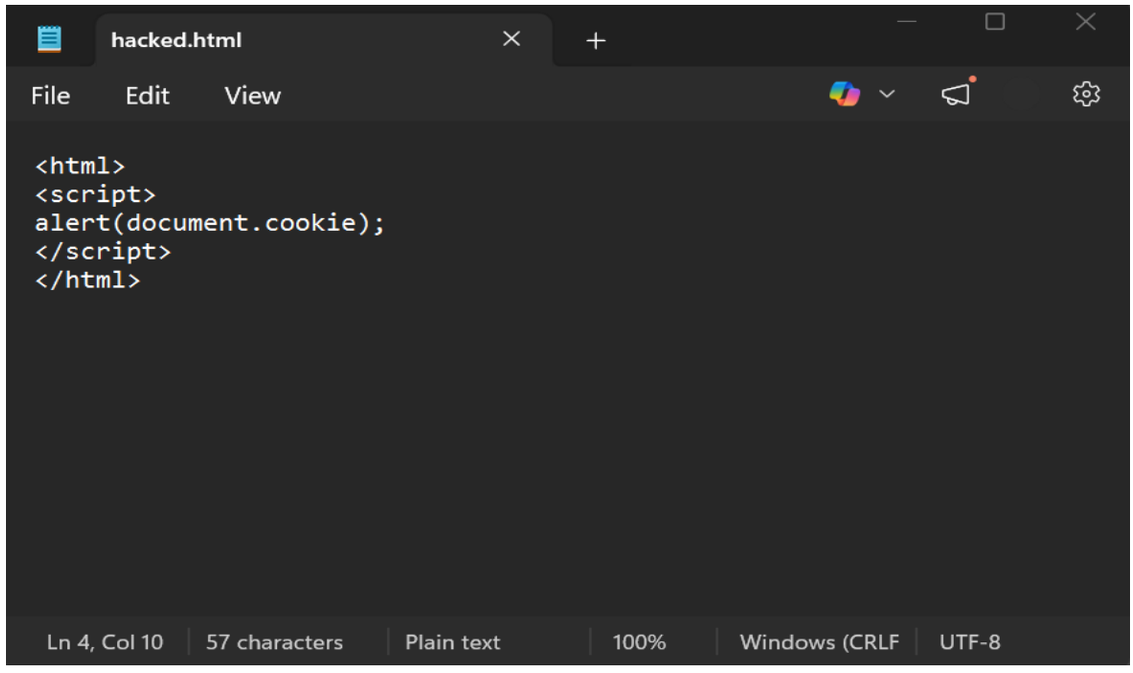
8.1	Information disclosure
Description	The file dump.gtl was directly accessible through the browser without any authentication or authorization check. It exposed sensitive application data including admin credentials stored in plaintext
Target	https://google-gruyere.appspot.com/
Risk rating	High
Attack technique	Direct URL enumeration by accessing dump.gtl directly through the browser using this link https://google-gruyere.appspot.com/367972211113895789518766501968514277425/dump.gtl with no prior authentication and the username is shown “admin” and password “admin” which I set when I was signing up.
Countermeasure	Always make sure debug features are not installed. In this case, delete dump.gtl. This is an example of the kind of debug feature that might be left in an application by mistake. If a debug feature like this is necessary, then it needs to be carefully locked down: only admin users should have access and only requests from debug IP addresses should be accepted.
Proof of concept	Screenshot below

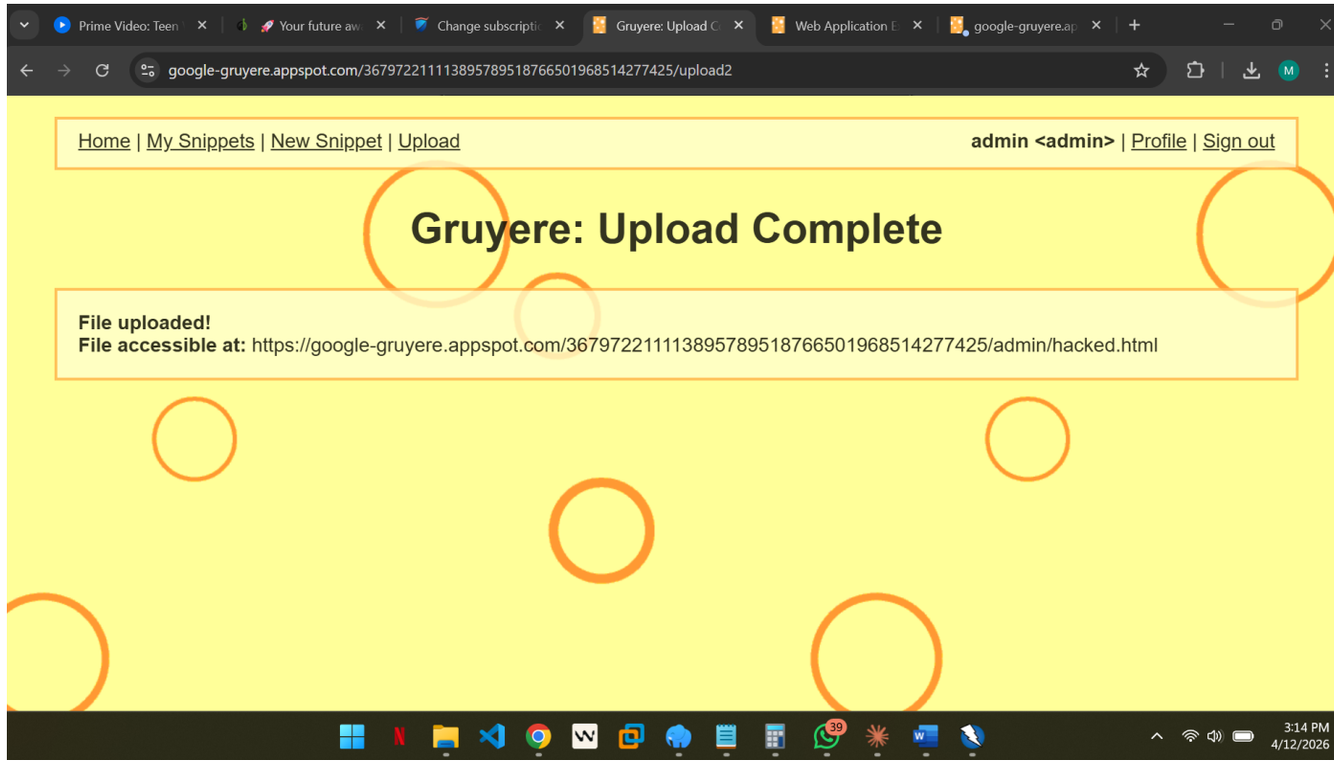


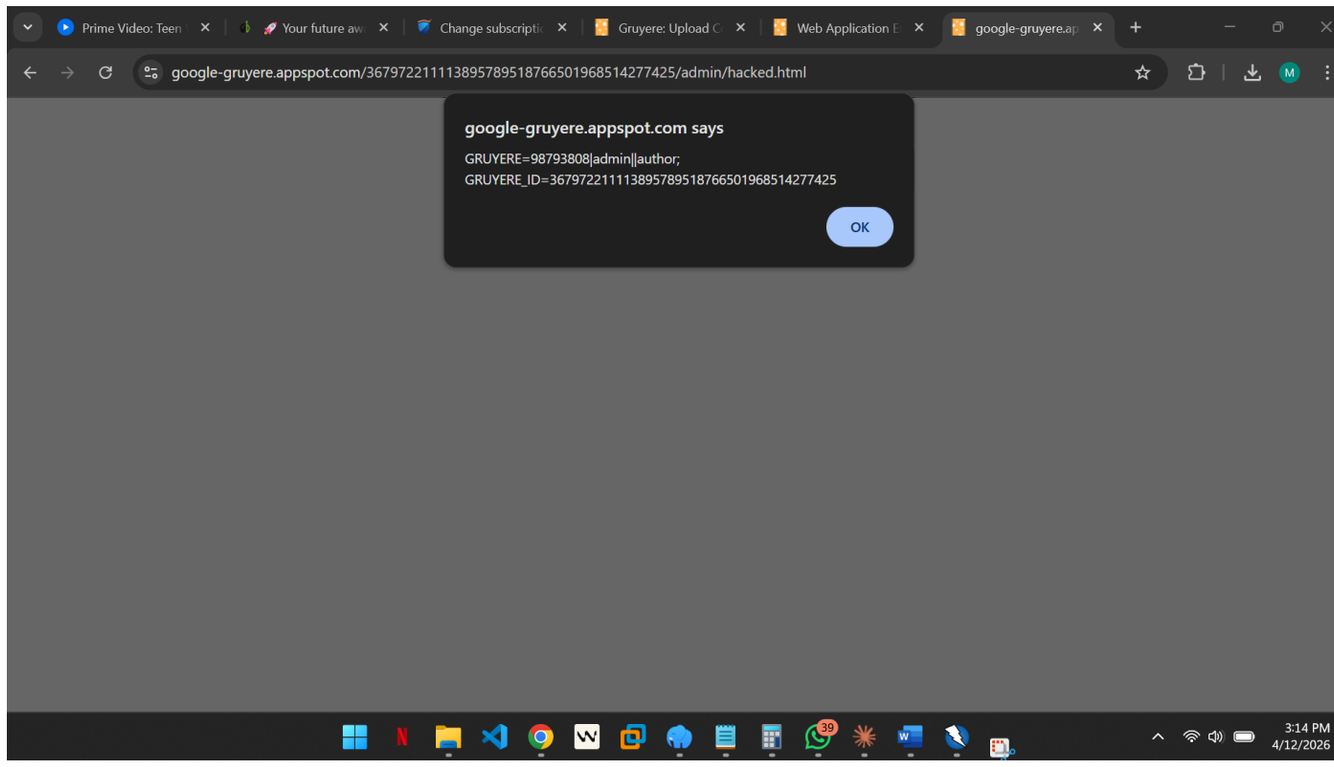
8.2	Stored xss (Cross site scripting)
Description	The application failed to sanitize or encode user-supplied input before rendering it in the browser. By injecting <script>alert(1)</script> into an input field, the malicious script was executed directly in the browser, confirming the presence of a Cross-Site Scripting.
Target	
Risk rating	High
Attack technique	I was able to identified an unsanitized input field(snippets field) on the page and injected a JavaScript payload <script>alert(1)</script> directly into it. The application rendered the input without any encoding or validation, causing the browser to execute the script and display an alert box confirming successful code execution
Counter measure	Validate and sanitize all user-supplied input server-side and implement content security policy header to block inline scripts.
Poof of concept	Screenshot below



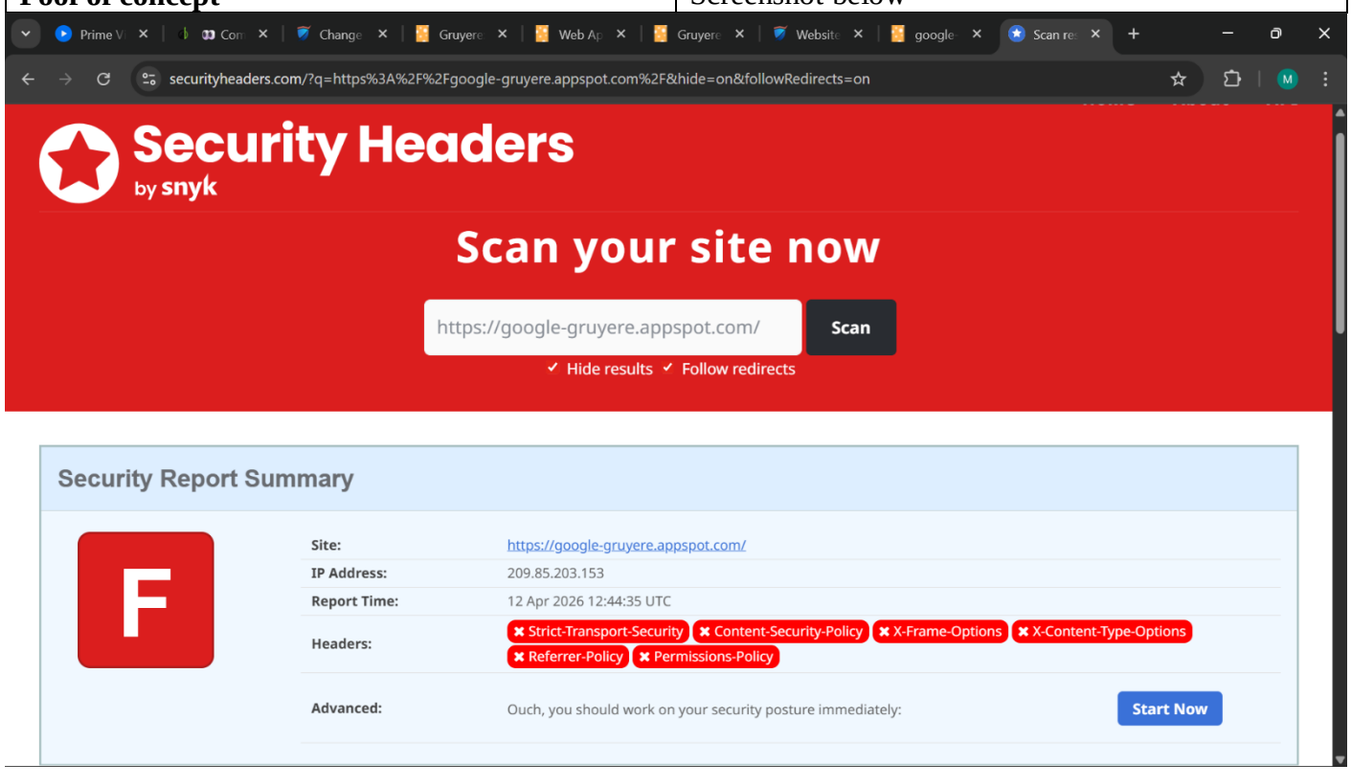
8.3	**File upload XSS**
Description	The website allowed unrestricted upload of HTML file (hacked.html) containing executable JavaScript without any content validation or file type restriction. This vulnerability is more severe because the malicious file is permanently stored on the server and accessible to anyone via a direct URL, making it persistently exploitable.
Target	
Risk rating	High
Attack technique	I crafted a malicious HTML file containing a JavaScript payload designed to extract session cookie information .The file was uploaded through the website file upload functionality without any server-side validation rejecting it. Upon successful upload the application generated a publicly accessible URL:https://google-gruyere.appspot.com/367972211113895789518766501968514277425/admin/hacked.html which when clicked revealed active session cookie data
Counter measure	The application should strictly restrict file uploads by whitelisting only safe, non-executable file types such as .jpg, .png, and .pdf, while rejecting any HTML, JavaScript, or SVG files that can carry executable scripts. All session cookies must be protected with the HttpOnly and Secure flags to prevent JavaScript from accessing them, and a strong Content-Security-Policy header should be enforced to block inline script execution across all pages.
Poof of concept	3 Screenshots below




google-gruyere.appspot.com/367972211113895789518766501968514277425/upload2
Home | My Snippets | New Snippet | Upload
admin <admin> | Profile | Sign out
Gruyere: Upload Complete
File uploaded!
File accessible at: https://google-gruyere.appspot.com/367972211113895789518766501968514277425/admin/hacked.html
3:14 PM
4/12/2026


google-gruyere.appspot.com/367972211113895789518766501968514277425/admin/hacked.html
google-gruyere.appspot.com says
GRUYERE=98793808|admin||author;
GRUYERE_ID=367972211113895789518766501968514277425
OK
3:14 PM
4/12/2026

8.4	Absence of security headers
Description	Securityheaders.com is a trusted site used to check if a website has security headers. The target website was scanned and the results show that no security header was set. absence of these headers the website is susceptible to clickjacking, scripts attacks (XSS)
Target	
Risk rating	medium
Attack technique	The target website was scanned using securityheaders.com site.
Counter measure	Security headers must implement like Strict-Transport-Security: max-age=31536000; to enforce encrypted HTTPS communication while a strong Content-Security-Policy header should be configured preventing the browser from executing malicious inline scripts that could compromise user sessions. Furthermore X-Frame-Options: SAMEORIGIN must be set to block unauthorized iframe embedding defending against clickjacking attacks, and X-Content-Type-Options: nosniff should be enforced to prevent MIME-sniffing ensuring the browser strictly adheres to declared content types reducing content injection risks.
Poof of concept	Screenshot below



```
+ Target Hostname:    google-gruyere.appspot.com
+ Target Port:        80
+ Platform:           Unknown
+ Start Time:         2026-04-12 10:30:53 (GMT-4)
---------------------------------------------------------------------------
+ Server: Google Frontend
+ Multiple IPs found: 172.217.170.212, 2a00:1450:401a:801::2014
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests ski
pped.
+ [999996] /robots.txt: contains 12 entries which should be manually viewed. See: https:/
/developer.mozilla.org/en-US/docs/Glossary/Robots.txt
+ [013587] /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-
Security
+ [013587] /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ [013587] /: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Optio
ns
+ [013587] /: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ [013587] /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
```

8.5	**Cross site request Forgery(CSRF)**
Description	The website was not implemented with any CSRF token or request verification mechanism on state-changing actions such as deleting snippets. During the assessment a delete action URL was intercepted using Burp Suite revealing a fully exposed endpoint deletesnippet?index=0 that executes the delete action when visited directly in the browser without any additional authentication or token validation.
Target	
Risk rating	High
Attack technique	I was able to intercept the HTTP response using Burp Suite and identified the following state-changing endpoint: https://google-gruyere.appspot.com/4087994693839216574613055669419947219 58/deletesnippet?index=0 When this URL was pasted directly into the browser and executed, the snippet at index 0 labeled "new snippet" was permanently deleted without any confirmation, token validation, or re-authentication, confirming successful CSRF exploitation. In a real attack scenario an attacker could embed this URL in a malicious page as an image tag or hidden form that silently executes when visited by an authenticated victim:
Counter measure	All state-changing actions such as delete, edit, and update must be protected with a unique unpredictable CSRF token that is validated server-side on every request, ensuring that only intentional user-initiated actions are processed.
Poof of concept	Screenshot below

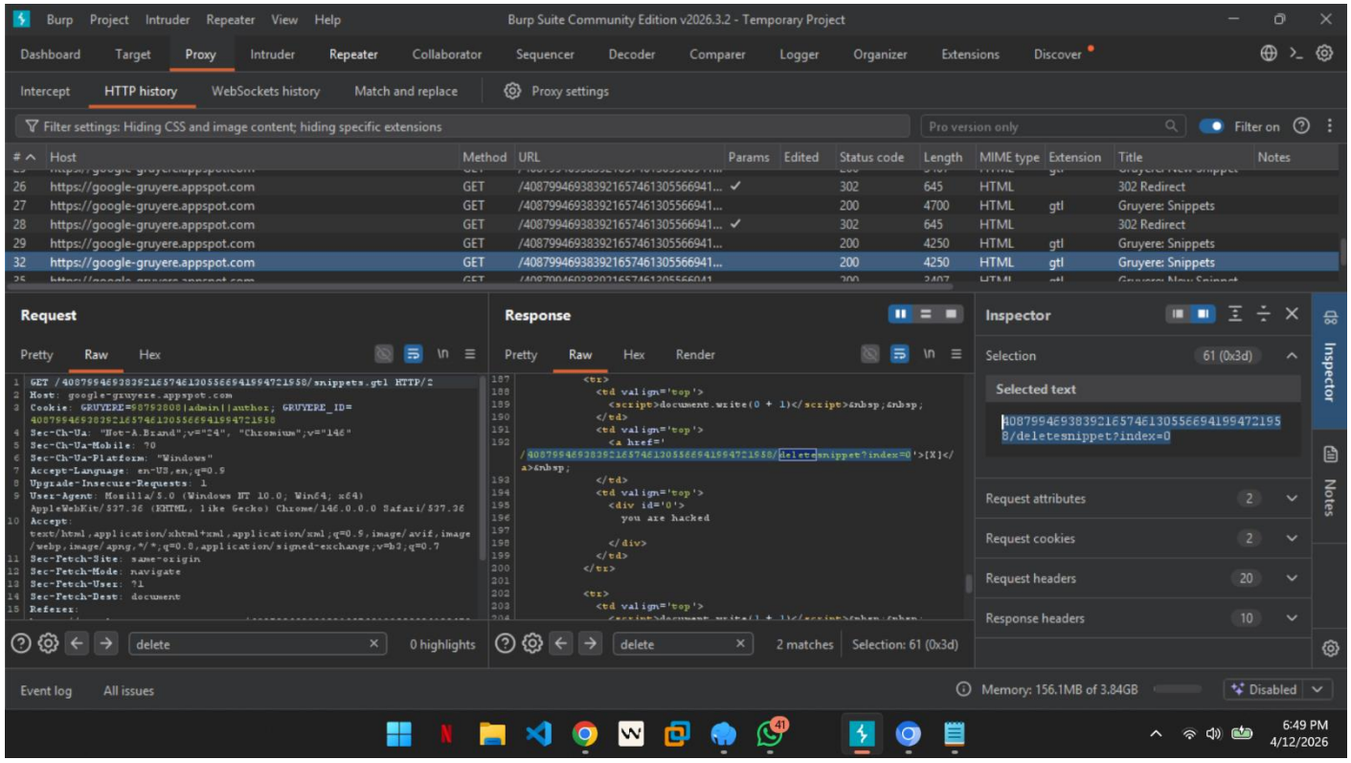


Figure: shows the obtained url that allows deletion of snippet with authentication.

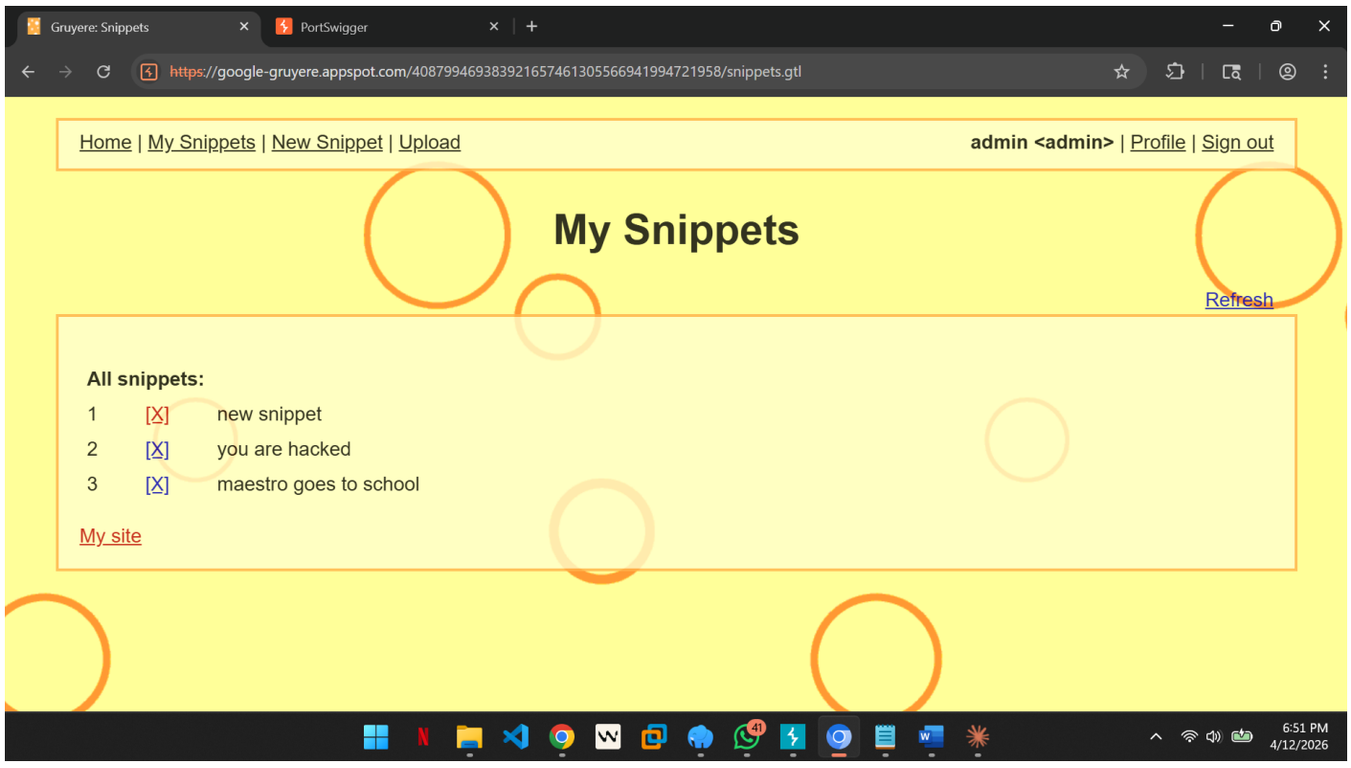


Figure: shows all the three snippets

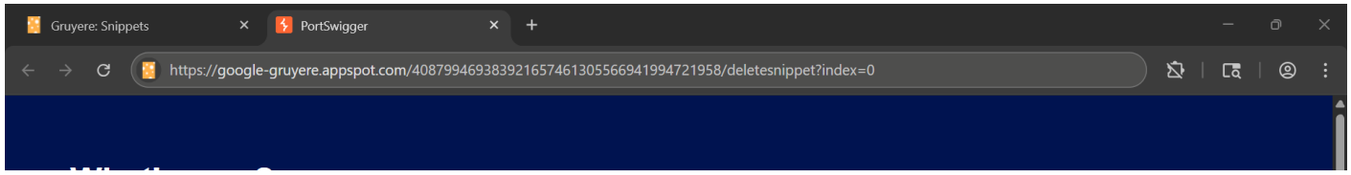


Figure: shows the url that deletes snippet with index 0 once clicked.

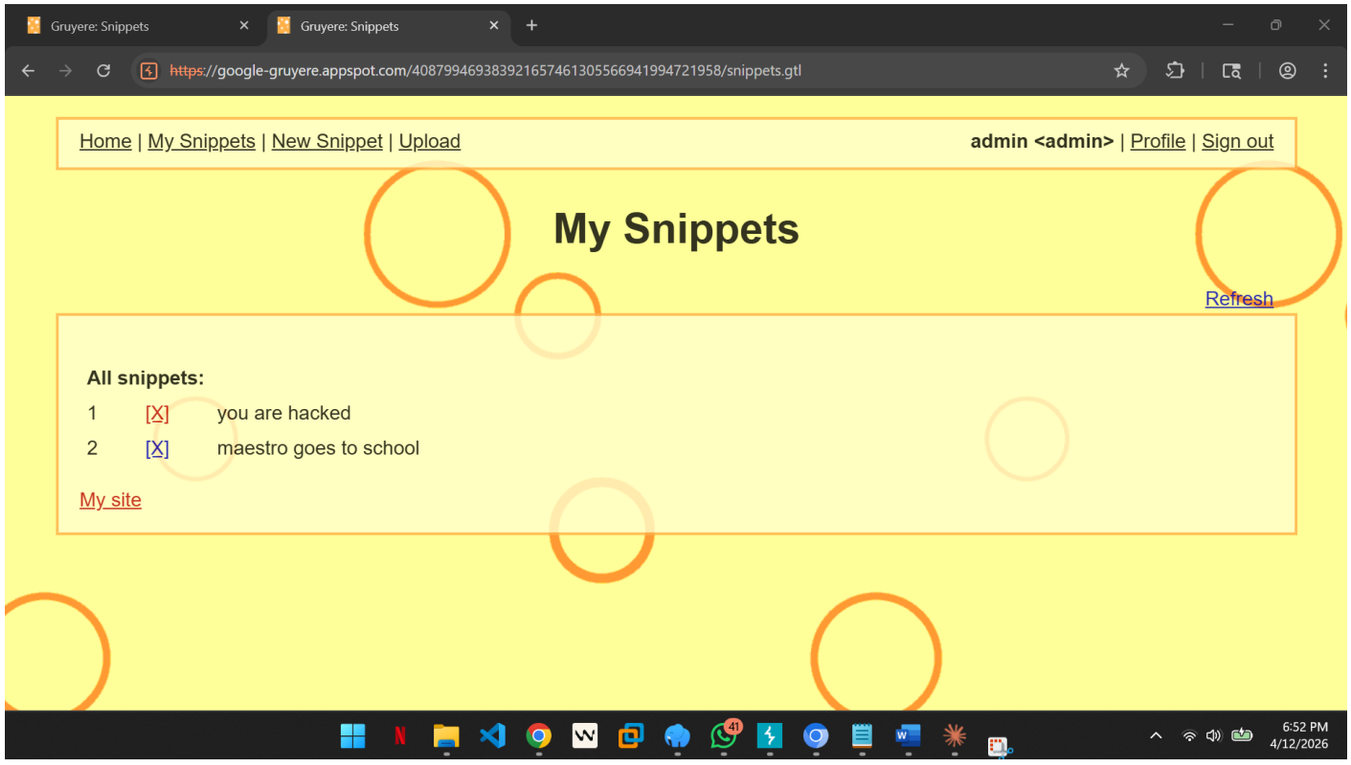


Figure: Confirms the deletion of snippet with index 0 which was "new snippet"

8.6	AJAX-Based Client-Side Manipulation
Description	The website uses AJAX to dynamically load and refresh content on the home and snippets pages without implementing proper server-side validation on the data returned. Since AJAX runs code entirely on the client side, the scripts responsible for handling requests and updating the DOM are fully visible and accessible to any attacker inspecting the page source, exposing the application logic and creating multiple opportunities for manipulation of requests and responses between the client and server.
Target	
Risk rating	medium
Attack technique	Intercept and modify AJAX requests sent between the browser and server using a proxy tool such as Burp Suite or OWASP ZAP, altering parameters, injecting malicious payloads, or manipulating response data before it is rendered in the browser.
Counter measure	
Proof of concept	Screenshot below shows That AJAX functionality was confirmed through a direct request to the feed.gtl endpoint, which returned a JSONP callback pattern _feed() proving that the application uses AJAX for dynamic content

	loading, while the response data contained an unencoded XSS payload onmouseover="alert(1)" demonstrating that raw HTML is returned in the JSON response without any output encoding, and the endpoint was found to be publicly accessible without any authentication, confirming that the AJAX endpoint is fully exposed to unauthenticated users.

```
Session   Actions   Edit   View   Help
┌──(maestro㉿kali)-[~]
└─$ curl -s http://google-gruyere.appspot.com/367972211113895789518766501968514277425/feed.gtl

_feed((

{
"private_snippet":
  ""
,"admin":
  "<a onmouseover="alert(1)" href="#">read this!</a>"

,"cheddar":
  "Gruyere is the cheesiest application on the web."
,"brie":
  "Brie is the queen of the cheeses<span style=color:red>!!!</span>"
}

))
```

9. APPENDIX

TOOLS	DESCRIPTION
Nmap	Service detection,vulnerability scanning
Owasp Zap	
Burpsuite	Vulnerability analysis
Securityheader.com	Security headers analysis